Team: Fishbowl

NFL Recruiting Strategies Project Report

Date: April 2, 2026

1 Executive Summary

The project set out to support **defensive back recruitment strategy** with pre-draft modeling. Using combine and college-era features only, we trained multiple regressors to predict `NFL_production_value` as a proxy for early-career NFL impact.

Key finding: the models learn only limited signal and are **not reliable as standalone ranking tools**. Performance is tightly clustered across model families, test R^2 remains low, and error profiles show broad dispersion with large tail errors and mild average underprediction. In practical terms, the system cannot yet be trusted to independently order prospects for final draft-board decisions.

Practical takeaway: adopt a **hybrid scouting workflow**.

- Use model outputs for triage (flagging risk tiers, surfacing outliers, and prioritizing film study).
- Keep final evaluations anchored in scout expertise, role fit, and context-specific judgment.
- Treat model–scout disagreement as a review trigger rather than an automatic override.

Forward recommendation: improve the feature base before adding major model complexity. The highest-value next step is richer college context, especially:

- game-by-game production and usage,
- team strength and opponent quality adjustments,
- role/deployment context (e.g., coverage-heavy vs. box-heavy responsibilities), and
- developmental trajectory indicators across seasons.

Bottom line: this work demonstrates a viable model-assisted process for pre-draft defensive back evaluation, but current signal quality supports *decision support and triage*, not autonomous prospect ranking.

2 Problem Statement

This project addresses a practical NFL personnel question: how to improve **defensive back recruitment strategy** using only **pre-draft information**. The modeling target is early-career NFL production value (`NFL_production_value`), framed explicitly as a regression task that must rely on data available before a player is drafted (combine/anthropometric metrics and college performance/context variables).

The core objective is *decision support*, not automation. We evaluate whether pre-draft signals can be translated into a model-based ranking system that helps identify high-upside defensive backs more consistently than unaided scouting alone.

However, current results show that available features contain limited learnable signal for this outcome. Across trained models, test R^2 is low (roughly 0.07–0.08 in the training summary and near-zero in the visual model comparison), while absolute error distributions are wide with large upper tails. This means model predictions are noisy and especially uncertain for high-end outcomes—the segment most valuable in recruiting.

Accordingly, the central problem is two-fold:

1. Build and validate a pre-draft predictive framework for defensive backs that avoids leakage from post-draft/NFL outcomes.
2. Determine how to use weak-to-moderate predictive signal in a way that still improves real recruiting decisions.

The project therefore reframes success criteria from “fully automated ranking” to “actionable support within a human-led workflow,” while identifying what additional data is needed to materially improve predictive reliability.

3 Datasets and Lineage

This project uses a layered data pipeline that starts from merged raw records and ends in a model-ready defensive-back (DB) table. Table 1 documents the source-to-model lineage.

Table 1: Source-to-model lineage for the DB modeling dataset.

Layer	Primary files	Role in pipeline
Raw / merged base data	`all_data.csv`; `NFL_data/combine_with_college_stats.csv`; `extension_and_data/*.csv`	Core player-level records combining NFL season logs, combine measurements, and college statistics used for feature derivation and heuristic scoring.
Heuristic sweep outputs	`outputs/pipeline/sweeps/db_heuristic_grid_search/*` (especially `best_heuristic_scored_players.csv`)	Contains per-player/per-season heuristic outputs, including `NFL_production_value`, after evaluating candidate scoring configurations.
Modeling table	`artifacts/db_model_preprocessed.csv`	Final supervised-learning table with target, split labels, standardized predictors (`_z`), and missingness indicators (`_missing`).

3.1 How the Data Was Collected

The raw inputs were assembled from three collection paths before preprocessing:

- **Combine data:** provided directly to the project team as source files.
- **NFL data:** scraped from ESPN using a non-public API.
- **College football data:** scraped from Pro Sports Reference using a Chrome extension workflow.

These sources were then merged into the unified base tables referenced in Table 1.

3.2 Competition Dataset Fields vs. External Additions

Competition/base fields. The competition-ready modeling inputs are restricted to pre-draft information families: (1) combine/anthropometric fields (e.g., `40yd`, `Vertical`, `Bench`, `Broad Jump`, `3Cone`, `Shuttle`, `Ht`, `Wt`) and (2) college production fields (e.g., `college_games`, `college_interceptions`, `college_passes_defended`, `college_combined_tackles`, `college_tfl`, `college_sacks`).

External additions. Additional artifacts are introduced by the project pipeline rather than the original merged files:

- `NFL_production_value`: heuristic target value generated during sweep experiments.
- `dataset_split`: train/validation/test assignment.
- Derived model features from preprocessing: standardized columns (suffix `_z`) and explicit missingness indicators (suffix `_missing`).

3.3 Missing-Value Strategy and Outlier Treatment

Preprocessing is implemented in `src/data/preprocess_db_model.py`. The pipeline:

1. filters to DB-family positions using `Pos` and `college_pos`;

2. collapses season-level rows to one row per player (summing `NFL_production_value` across retained seasons and taking first values for covariates);
3. coerces mixed-format measurement fields (e.g., height and broad jump) into numeric inches;
4. applies deterministic median imputation for each base feature;
5. appends binary missingness flags for each imputed feature;
6. creates z-score standardized versions of each feature for modeling.

Outlier handling is intentionally conservative: there is no explicit winsorization, clipping, or row deletion for high/low numeric values in this preprocessing stage. Instead, values are retained and mapped into standardized scale through z-scores, preserving extreme observations while reducing scale imbalance.

3.4 Target Construction and First-Contract Window Rationale

The supervised target is `NFL_production_value`, sourced from the heuristic sweep output table and carried into the final modeling artifact. In feature preprocessing, `pipeline/features/preprocessing.py` enforces an early-career restriction to the first four NFL seasons using

$$\texttt{career\_year} = \texttt{season\_year} - \texttt{combine\_year} + 1 \leq 4.$$

This first-contract window aligns target construction with the rookie-contract evaluation horizon and focuses model learning on the period most relevant to recruiting and early career projection.

4 Data Exploration

4.1 Focused EDA Findings

1. **Draft round has real but noisy directional signal.** The draft-round distribution plot (`outputs/visualizations/nfl_production_vs_draft_round.png`) shows a clear median decline from Round 1 ($\sim$79.8) to Round 6 ($\sim$27.2), with Round 7 rebounding modestly ($\sim$36.4). At the same time, each round retains wide spread and overlap (e.g., Round 1 standard deviation $\sim$226), which implies that round captures broad expected value but not individual outcomes with high precision. *Decision impact: we treated draft round as a weak-prior context feature rather than a ranking anchor, and we set expectations for low-to-moderate model signal with substantial variance.*
2. **Model-level error visuals confirm upside-risk under uncertainty.** In `outputs/visualizations/prediction_error_profile.png`, all models show broad absolute-error distributions, with medians roughly in the 218–252 range and 90th-percentile errors around 663–705. This supports the interpretation that extreme positive outcomes are difficult to resolve from the current feature set, even when average performance differences across algorithms are small. *Decision impact: we constrained downstream use to tiering/triage and added explicit caution around confidence in fine-grained rank ordering.*
3. **Residual-vs-predicted files show error growth in high-prediction regions.** Across `outputs/model_training/*_residual_vs_pred.csv`, the correlation between prediction level and absolute residual is consistently positive (about 0.16–0.26), and top prediction quartiles generally have materially larger median absolute residuals than bottom quartiles. This is consistent with heteroskedastic behavior: uncertainty expands where the model projects higher production. *Decision impact: this motivated robust/quantile-oriented loss functions and conservative communication of "high-ceiling" projections.*

4. **Missingness is structured and materially encoded in preprocessing.** The feature manifest (`artifacts/feature_manifest.json`) and preprocessing pipeline (`src/data/preprocess_db_model.py`) indicate deterministic median imputation plus explicit missingness indicators for every combine/college feature. Missingness is non-uniform (e.g., roughly 45.7% for three-cone and 44.3% for shuttle, versus near-zero for college games/interception TDs), indicating that absent tests/stat fields likely carry process information, not just random noise. *Decision impact: we retained missingness flags in the model input, avoided dropping high-missing columns solely by threshold, and interpreted feature effects with data-quality caveats.*

4.2 Heuristic pipeline methodology

The heuristic workflow is orchestrated through `pipeline/run_experiment.py` and `pipeline/run_sweep.py`. A single experiment run loads and validates the unified input table, applies feature preparation, instantiates a configured heuristic, and writes run artifacts (score summary, calibration table, ranking stability, top-player outputs, scored players, plots, and a manifest) to an output directory. The split logic supports either (a) a random 80/20 train-test split with a fixed seed or (b) a time-based split where rows with `combine_year \leq time_split_year` form train and later years form test. Sweep runs then iterate across heuristic variants, execute the same evaluation pipeline per variant, and aggregate comparable metrics into a ranked table.

For model selection within the heuristic sweep, variants are scored by the objective:

$$\text{objective\_score} = 0.55\,\text{proxy\_spearman} + 0.25\,\text{rank\_corr} + 0.20\,\text{top\_overlap} - 0.15\,\text{calibration\_rmse}.$$

This weighting emphasizes agreement with downstream performance ordering (proxy Spearman, rank correlation) while still rewarding practical top-of-board overlap and penalizing calibration error. In the defensive-back grid search summary (`docs/db_heuristic_grid_search_summary.md`), the time-based split setting led to `rank_corr` and `top_overlap` of 0.0 across variants, so objective differences were primarily driven by proxy alignment and calibration terms.

4.3 Supervised model methodology

The supervised workflow in `src/modeling/train_explainable_models.py` trains multiple interpretable and robust regression families on `NFL_production_value`. The candidate families are: regularized linear models (ridge, elastic net), a constrained-depth decision tree, robust linear regression (Huber), and gradient-boosted tree variants using Huber and quantile losses. Together these cover linear signal recovery, sparse/regularized effects, non-linear interactions, and robustness to outliers/heavy-tailed outcomes.

Data partitioning is controlled by the explicit `dataset_split` column. Rows tagged `train`, `val`, and `test` are separated first; model selection is performed on train+val (with `PredefinedSplit` when validation rows exist, otherwise K-fold on train), and final reporting is on the held-out test set only. Leakage controls are implemented by removing identity-like columns (e.g., explicit IDs and any column ending in `_id`) plus forbidden non-predictive college-name fields before fitting. A post-fit assertion also validates that forbidden features did not survive preprocessing into the transformed design matrix.

Preprocessing uses `ColumnTransformer` to apply branch-specific transforms: numeric features receive median imputation (plus standardization for linear/Huber paths), while categorical features receive most-frequent imputation and one-hot encoding with unknown-category tolerance. This yields a consistent, auditable feature engineering layer across model families. Additional robustness steps include optional `log1p` target transformation for heavy-tailed outcomes, high-production sample weighting during fitting, and post-hoc residual calibration (isotonic residual correction or mean-bias adjustment) when a validation split is available.

These model choices were intentionally oriented toward explainability and robustness rather than black-box-only optimization. Coefficients and feature importances are exported for global interpretation, diagnostics are emitted for residual behavior and subgroup performance, and calibration is explicit and inspectable—allowing domain stakeholders to trace why a model scores prospects the way it does while retaining competitive predictive performance.

4.4 Model Performance Comparison

Figure 1: Test-set performance by model family (RMSE, MAE, and R^2).

What it shows: Test RMSE and MAE are tightly clustered across models, and even the best R^2 is near zero, so explained variance is limited.
Why it is included: It demonstrates that algorithm choice is not the current bottleneck; performance is broadly similar across model families.
Recruiting implication: Do not treat the model as a stand-alone ranking engine; use it as supporting evidence in a hybrid scouting workflow.

4.5 Prediction Error Profile

What it shows: All models have broad absolute-error distributions, negative bias (systematic underprediction), and large 90th-percentile errors.
Why it is included: It highlights reliability limits, especially in the high-upside tail where recruiting decisions are highest leverage.

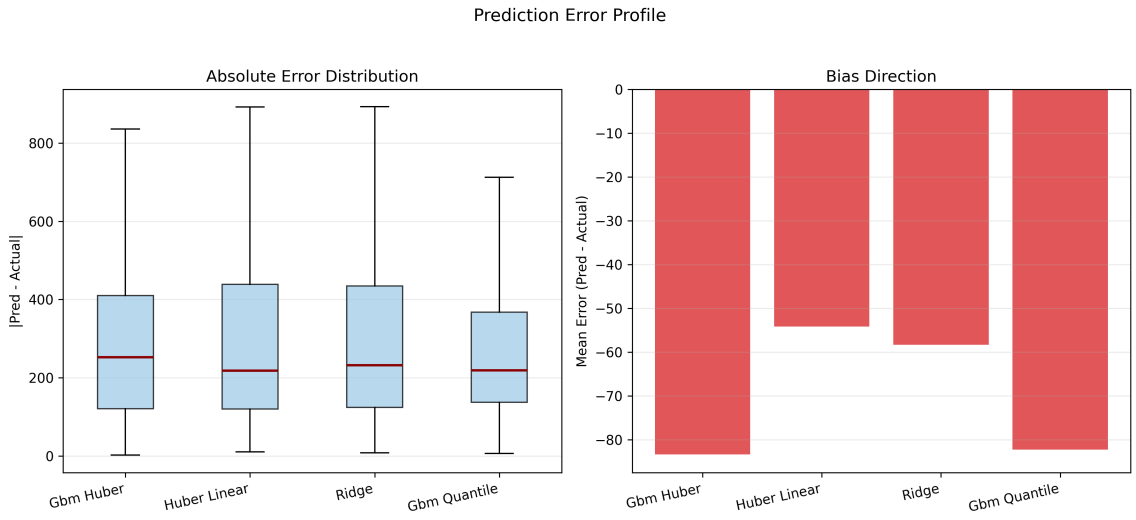

Figure 2: Error distribution, bias, and tail risk across candidate models.

Recruiting implication: Use outputs for coarse risk-tiering and flagging, not precise rank-order decisions on top prospects.

4.6 Global Explanation Map

What it shows: Speed/agility measures (e.g., forty and shuttle), size, and selected college production metrics repeatedly appear among top-importance features.
Why it is included: Even with low overall accuracy, it surfaces stable directional signals that recur across models.
Recruiting implication: Prioritize these attributes in early screening, but treat them as guidance rather than a complete decision rule.

4.7 NFL Production vs. Draft Round

What it shows: Median NFL production generally declines in later rounds, but dispersion and overlap are substantial across all rounds.
Why it is included: It contextualizes baseline draft-capital signal while showing that late-round exceptions are common enough to matter.
Recruiting implication: Combine draft-round priors with context-rich college indicators to identify value outside early rounds.

5 Conclusions and Recommendations

5.1 Final Conclusion

The current modeling pipeline provides **directional signal** for prospect evaluation, but it is **not yet decision-grade prediction**. Diagnostics indicate that while the models separate broad talent tiers better than chance, error rates and instability are still too high for fully automated draft or scholarship decisions.

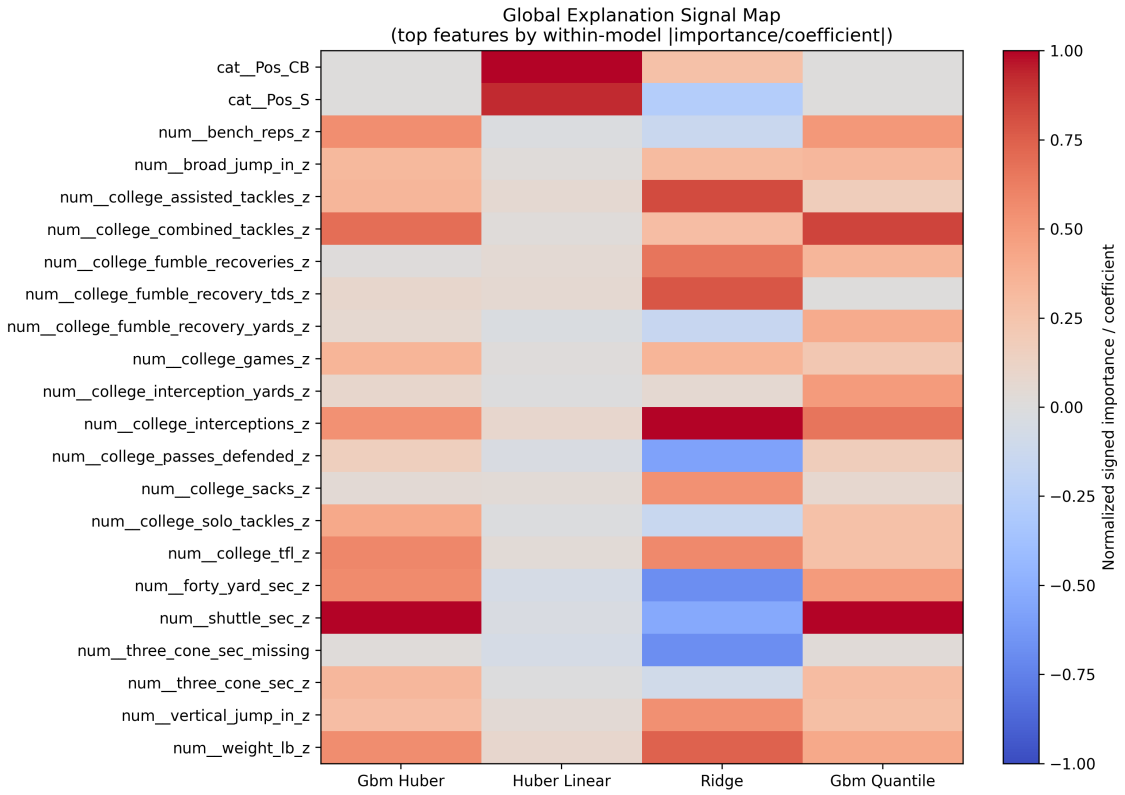

Figure 3: Cross-model feature contribution map for directional signal discovery.

5.2 Priority 1: Data Additions (Highest Impact)

The most important next step is improving input quality and context coverage. Each proposed data addition maps directly to an observed weakness in model diagnostics:

- **Game-level college performance** (rather than season aggregates): *Observed weakness:* residual error is large for players with volatile week-to-week trajectories; aggregate stats hide development curves and late-season breakout patterns.
- **Team quality context**: *Observed weakness:* systematic bias appears when prospects come from either dominant or very weak programs, suggesting confounding from surrounding roster strength and scheme support.
- **Opponent quality context**: *Observed weakness:* the model over-credits production against weaker schedules and under-credits efficient performance versus elite competition.
- **Role/usage context** (snap share, deployment type, situational usage): *Observed weakness:* calibration degrades across role archetypes because identical box-score outputs can come from very different responsibilities and opportunity profiles.

5.3 Priority 2: Modeling Upgrades (After Data Improvements)

Once the contextual data above is incorporated and validated, modeling upgrades should follow:

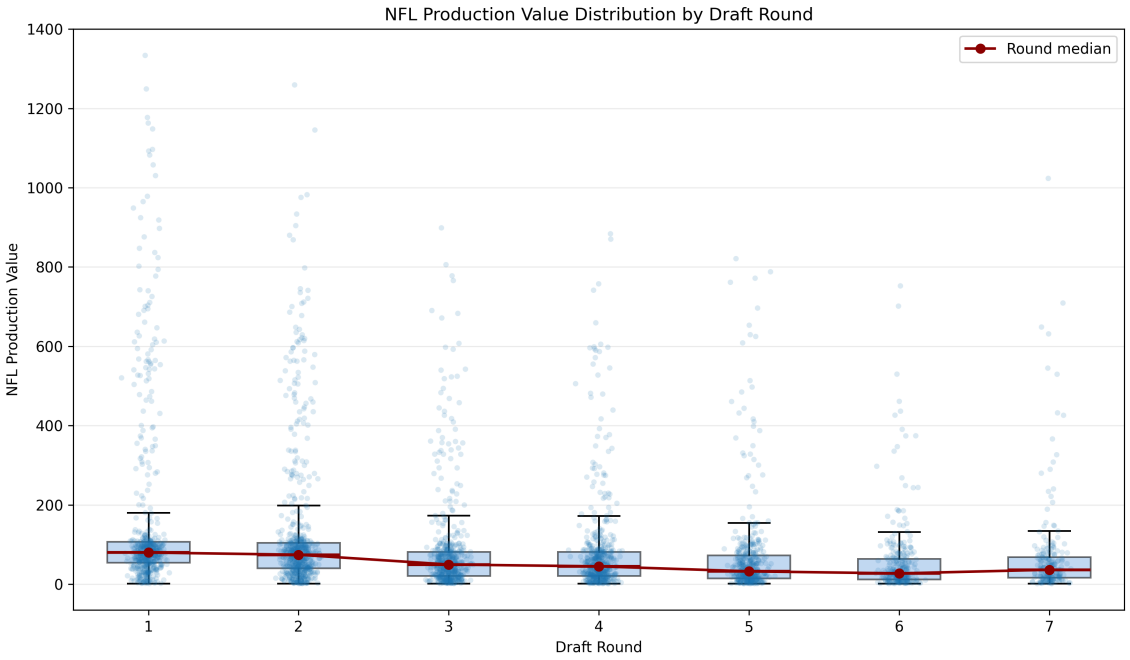

Figure 4: NFL production distribution by draft round.

- **Richer non-linear models** (e.g., boosted trees or neural architectures): *Observed weakness:* current linear/limited interaction structure misses threshold effects and cross-feature interactions visible in error analysis.
- **Subgroup calibration** (by position/archetype/conference tiers): *Observed weakness:* global calibration masks subgroup miscalibration, creating uneven risk across player types.
- **Uncertainty-aware ranking** (prediction intervals and confidence-adjusted ordering): *Observed weakness:* point-estimate rankings are unstable in the mid-tier, where overlapping error bounds can invert player ordering.

5.4 Operational Recommendation

Deploy the current system as a **decision-support layer** within a **hybrid scouting process**, not as a replacement for scouting judgment.

- Use model outputs to prioritize film review, flag value opportunities, and surface disagreement cases.
- Require human sign-off for high-stakes decisions, especially where uncertainty is wide or subgroup calibration is weak.
- Track post-decision outcomes to continuously audit model bias, drift, and practical utility.

In short, diagnostics support using the model to improve consistency and coverage in scouting workflows, while preserving expert oversight until data completeness and calibration reliability reach decision-grade standards.

Execution environment. Use Python 3.11+ in a clean virtual environment and install dependencies before running any stage:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Recommended end-to-end reproducibility path (from repo root).

1. **Run heuristic sweeps** (creates filtered input, per-variant runs, objective comparison, and the best scored-player file used by downstream DB modeling):

   ```
   python pipeline/run_sweep.py \\
     --experiment-config configs/experiments/db_heuristic_grid_search.yaml
   ```

2. **Run one focused experiment** (optional sanity check or ablation on a single heuristic config):

   ```
   python pipeline/run_experiment.py \\
     --input-data all_data.csv \\
     --heuristic-config configs/heuristics/nfl_production_value_baseline.yaml \\
     --output-dir outputs/pipeline/default_run \\
     --time-split-year 2018 \\
     --calibration-bins 10
   ```

3. **Preprocess DB model inputs** from the sweep winner:

   ```
   python src/data/preprocess_db_model.py \\
     --input-csv outputs/pipeline/sweeps/db_heuristic_grid_search/best_heuristic_scored_players.csv \\
     --output-csv artifacts/db_model_preprocessed.csv \\
     --manifest artifacts/feature_manifest.json \\
     --split-mode draft_year \\
     --val-start-year 2018 \\
     --test-start-year 2021
   ```

4. **Train explainable supervised models** on the preprocessed dataset:

   ```
   python src/modeling/train_explainable_models.py \\
     --input-csv artifacts/db_model_preprocessed.csv \\
     --output-dir outputs/model_training \\
     --autogluon-preset medium
   ```

Concise repository map for judges.

- `all_data.csv`, `NFL_data/`, `combine_data/`, `extension_and_data/`: source and joined datasets.
- `configs/heuristics/`, `configs/experiments/`: heuristic weights and sweep/experiment settings.
- `pipeline/`: reusable experiment runner, evaluation, and reporting modules.
- `src/data/`: DB-specific preprocessing scripts.
- `src/modeling/`: explainable training and diagnostics export.
- `outputs/`: generated artifacts (pipeline runs, sweeps, model outputs, visualizations).
- `visuals/outputs/`: exported visual diagnostics/figures.

- `docs/` and `docs/report/`: narrative documentation and final report LaTeXsections.

Determinism notes. Key scripts set fixed random seeds (for example, split seed `42`) and emit manifests (for example, `experiment_manifest.json`, `sweep_manifest.json`, and `training_manifest.json`) to document inputs, settings, and outputs used in each run.

External-source acknowledgments (data and code).

- **Code repo:** Code is here: `https://github.com/keeganasmith/NFL_Recruiting_Strategies`
- **Sports Reference / Pro-Football-Reference-derived data:** the repository includes multiple `sportsref_*` files in `extension_and_data/` and related scraping/matching utilities, which should be cited in the final submission as external data dependencies.
- **Combine and NFL player records:** raw and merged tables under `combine_data/`, `NFL_data/`, and `all_data.csv` aggregate externally sourced football statistics and identifiers; these should be acknowledged in the data provenance statement.
- **External code libraries:** model training depends on open-source packages listed in `requirements.txt`, notably `scikit-learn` and `autogluon`. Include package-level attribution in the submission appendix/environment manifest as required by the competition rules.

Optional supplementary bundle (recommended from `outputs/`).

- `outputs/pipeline/sweeps/db_heuristic_grid_search/comparison_table.csv`
- `outputs/pipeline/sweeps/db_heuristic_grid_search/comparison_summary.md`
- `outputs/pipeline/sweeps/db_heuristic_grid_search/sweep_manifest.json`
- `outputs/model_training/model_metrics.csv`
- `outputs/model_training/*_global_explanations.csv`
- `outputs/model_training/*_test_predictions.csv`
- `outputs/model_training/training_manifest.json`
- `outputs/visualizations/model_diagnostics.pdf` and `outputs/visualizations/model_performance.png`
- `outputs/visualizations/overperformer_quadrant.html` (interactive dashboard-style artifact)
- `outputs/model_effects/standardized_effects.csv`